

Topic # "Dynamic Programming"

#: What is dynamic programming?
Developed by Richard Bellman in 1950s.

Dynamic programming, like the divide and conquer method, solves problems by combining the solutions to subproblems.

2 A dynamic-programming algorithm solves each subproblem just once and then saves its answer in a table, thereby avoiding the work of recomputing the answer every time it solves each subproblem.

Typically dynamic programming is apply to optimization problems.

Optimization problem mean that when we are trying to find out the minimum or maximum solution of a problem.

The dynamic programming guarantees to find the optimal solution of a problem if the solution exist.

features of dynamic programming:-
it is applicable for the following problems:-

- Overlapping subproblem
- Optimal substructure

Q# What Steps should be follow when we develop a dynamic-programming algorithm

- 1) Characterize the structure of an optimal solution
- 2) Recursively define the value of an optimal solution.
- 3) Compute the value of an optimal solution, typically in bottom-up fashion.
- 4) Construct an optimal solution from computed information.

Q# How does the dynamic programming approach work?

- => It breaks down the complex problem into simpler subproblem.
- => it finds the optimal solution these problems.
- => it stores the results of subproblems. The process of storing the results of subproblems is known as memorization.
- => it reuses them so that same subproblem is not calculated more than once.
- => Finally, Calculate the result of the main problem.

Elements of dynamic Programming

The elements of dynamic programming are as follows:-

- Overlapping sub problem
 - ⇒ Top down approach
 - ⇒ Bottom up approach
- Optimal substructure

Overlapping Sub problem:-

⇒ DP Split the problem into sub problem.

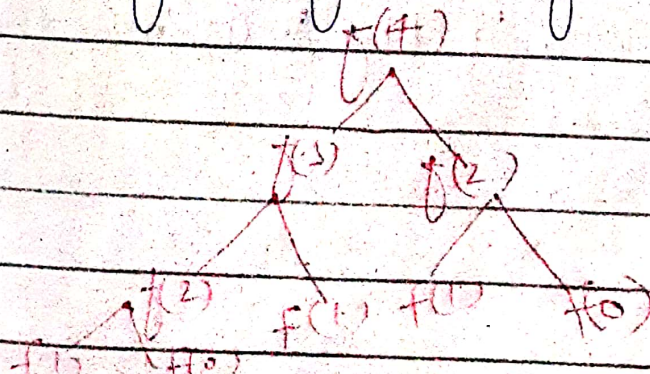
⇒ The overlapping subproblem is found in that problem where bigger problems share the same smaller problem.

⇒ In simple words; sub problems are repeated.

example:

Fibonacci Series

$$f(n) = f(n-1) + f(n-2)$$



Q# What is Optimal substructure

$\Rightarrow$ Consider a problem P , broken down into two subproblems, P_1 and P_2 .

$\Rightarrow$ we must be able to efficiently combine optimal solutions to P_1 and P_2 into an optimal solution P .

$\Rightarrow$ The structure of an optimal solution must contain optimal solutions to the recursive subproblems.

$\Rightarrow$ In simple words, optimal solution of the given problem can be obtained by finding the optimal solutions of all the sub-problems.

example

if we given a complex problem then we will break this problem into simpler problems optimally to find optimal solution of the given complex problem.

$$f(n) = f(n-1) + f(n-2)$$

Topic # LCS

Longest Common Subsequence

The LCS problem is finding the longest sequence which exists in both the given strings. To find DNA Subsequence:

Let us consider a sequence $S = \langle s_1, s_2, s_3, \dots, s_n \rangle$

A sequence $Z = \langle z_1, z_2, \dots, z_n \rangle$ over Σ is called a subsequence of S iff it can be derived from S by deletion of some elements.

Common Subsequence:

Suppose, X and Y are two sequences over a finite set of elements.

Then Z is a common subsequence of X and Y , if Z is a subsequence of both X and Y .

Longest Common Subsequence:

⇒ The longest subsequence that is common to all the given sequences, provided that the elements of the subsequence are not required to occupy consecutive positions within the original sequence.

Possible subsequence = 2^n

Optimal substructure of LCS:

$\Rightarrow$ Let $X = \{x_1, x_2, \dots, x_n\}$ and $Y = \{y_1, y_2, \dots, y_m\}$ be sequences, and let $Z = \{z_1, z_2, \dots, z_k\}$

be any LCS of X and Y .
1) if $x_m = y_n$ then $z_k = x_m = y_n$ and Z_{k-1} is an LCS of X_{m-1} and Y_{n-1} .

2) if $x_m \neq y_n$ then $z_k \neq x_m$ implies that Z is an LCS of X_{m-1} and Y .

3) if $x_m \neq y_n$, then $z_k \neq y_n$ implies that Z is an LCS of X and Y_{n-1} .

Recursive formula:

$$C[i, j] = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ C[i-1, j-1] + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j \\ \max(C[i, j-1], C[i-1, j]) & \text{if } i, j > 0 \text{ and } x_i \neq y_j \end{cases}$$

- A brute force approach we find all the subsequences of X and check each subsequence.

Algorithm:-

LCS - Length (X, Y)

$m = X.Length$

$n = Y.Length$

let $b[1..m, 1..n]$ and $c[0..m, 0..n]$
new tables

for $i = 1$ to m

$c[i, 0] = 0$

for $j = 0$ to n

$c[0, j] = 0$

for $i = 1$ to m

for $j = 1$ to n

if $x_i == y_j$

$c[i, j] = c[i-1, j-1] + 1$

$b[i, j] = \nwarrow$

else if $c[i+1, j] \geq c[i, j+1]$

$c[i, j] = c[i+1, j]$

$b[i, j] = \downarrow$

else

$c[i, j] = c[i, j+1]$

$b[i, j] = \rightarrow$

Return c and b

Y J C (1,5)

		0	1	2	3	4	5	6
	X	B	D	C	A	B	A	
0	0	0	0	0	0	0	0	
1	A	0	1	1	0	1	1	
2	B	0	1	1	1	2	2	
3	C	0	1	1	2	2	2	
4	B	0	1	1	2	2	3	
5	D	0	1	2	2	2	3	
6	A	0	1	2	2	3	4	
7	B	0	1	2	2	3	4	

Y B 1 2 3 4 5 6

	0						
1	↑	↑	↑	↑	←	←	
2	↖	←	←	←	↖	←	
3	↑	←	↖	←	←	←	
4	↖	←	↑	←	↖	←	
5	↑	↖	←	←	↑	←	
6	↖	↑	←	↖	←	↖	
7	↖	↑	←	↑	↖	←	

Time Complexity

Ⓐ (mn) Length

Ⓞ (m+n) Point

Print LCS.

Print - LCS(b, x, i, j)

if $i == 0$ or $j == 0$ — 1
return

if $b[i, j] == "x"$ — 1

Print - LCS(b, x, i-1, j-1)

Print x

else if $b[i, j] == "y"$

Print - LCS(b, x, i-1, j)

else

Print - LCS(b, x, i, j-1)

0

• Topical II Optimal binary search

⇒ An optimal BST is a BST, which has minimal expected cost of loc each node.

⇒ Search time of an element in is $O(n)$, whereas in balanced BST, search time is $O(\log n)$.

⇒ The search time can be improved in optimal binary search tree, place the most frequently used data at the root and closer to the root element, while placing the least frequently used data near leaves and in leaf.

⇒ Problem of an optimal BST can be rephrased as:

Given a list of n keys ($A_1, A_2, \dots, A_n$) and their frequencies of access ($F_1, F_2, \dots, F_n$), construct an optimal binary search tree in which the cost of search is minimum.

Formula for calculating the number of nodes:

$$\therefore \frac{2^n C_n}{n+1}$$

⇒ it may not have the lowest height it needs 3 tables to record probabilities, cost and root.

Algorithm:-

Optimal - BST (P, q, n)

Let $e[1 \dots n+1, 0 \dots n]$, $w[1 \dots n+1, 0 \dots n]$ and $root[1 \dots n, 1 \dots n]$ be new tables

for $i = 1$ to $n+1$

$e[i, i-1] = q_{i-1}$

$w[i, i-1] = q_{i-1}$

for $l = 1$ to n

for $i = 1$ to $n-l+1$

$j = i+l-1$

$e[i, j] = \infty$

$w[i, j] = w[i, j-1] + p_j + q_j$

for $x = i$ to j

$t = e[i, x-1] + e[x+1, j] + w[i, j]$

if $t < e[i, j]$

$e[i, j] = t$

$root[i, j] = x$

return e and $root$

Time complexity:- $\Theta(n^3)$

Recursive formulation:-

$$e[i, j] = \begin{cases} q_{i-1} & \text{if } j = i-1 \\ \min_{i \leq x \leq j} \{e[i, x-1] + e[x+1, j] + w[i, j]\} & \text{if } j \geq i \end{cases}$$

i	0	1	2	3	4	5
P_i		0.15	0.10	0.05	0.10	0.20
$q_{i, \text{DAY}}$	0.05	0.10	0.05	0.05	0.05	0.10

$i = 2, 3$ for $i = 1$ to 6

$$w[1,0] = e[1,0] = q_{1-1} = q_0 = 0.05$$

$$w[2,1] = e[2,1] = q_{2-1} = q_1 = 0.10$$

$$w[3,2] = e[3,2] = q_{3-1} = q_2 = 0.05$$

$$w[4,3] = e[4,3] = q_{4-1} = q_3 = 0.05$$

$$w[5,4] = e[5,4] = q_{5-1} = q_4 = 0.05$$

$$w[6,5] = e[6,5] = q_{6-1} = q_5 = 0.10$$

Iteration 1

for $l = 1$ to 5

for $i = 1$ to $n-l+1$

for $i = 1$ to $5-l+1$
 $i = 1$ to 5

$$j = i+l-1 \Rightarrow j = 1$$

$l = 1$

$i = 1$ to 5

$j = 1$
 $j = 2, 3, 4$

$$e[1,1] = \infty$$

$$w[1,1] = w[1,0] + P_1 + q_1 = 0.05 + 0.15 + 0.10 = 0.30$$

$$w[2,2] = w[2,1] + P_2 + q_2$$

$$= 0.10 + 0.10 + 0.05 = 0.25$$

$$w[3,3] = w[3,2] + P_3 + q_3$$

$$= 0.05 + 0.05 + 0.05 = 0.15$$

$$w[4,4] = w[4,3] + P_4 + q_4$$

$$= 0.05 + 0.10 + 0.05 = 0.20$$

$$w[5,5] = w[5,4] + P_5 + q_5$$

$$= 0.05 + 0.20 + 0.10 = 0.35$$

$i=1, j=1$
 for $x=1$ to 1
 $t = e[1,0] + e[2,1] + w[1,1]$
 $t = 0.05 + 0.10 + 0.30 = 0.45$

if $0.45 < \infty$ True

$$e[i,j] = t$$

$$e[1,1] = 0.45$$

$$\text{root}[1,1] = 1$$

$i=2$
 for $x=2$ to 2
 $t = e[2,1] + e[3,2] + w[2,2]$
 $t = 0.45 + 0.05 = 0.50 + 0.25 = 0.75$

$$0.35 < \infty \quad \text{True}$$

$$e[2,2] = 0.35$$

$$\text{root}[2,2] = 2$$

$i=3$
 for $x=3$ to 3
 $t = e[3,2] + e[4,3] + w[3,3]$
 $t = 0.05 + 0.05 + 0.15 = 0.25$

$$0.25 < \infty \quad \text{True}$$

$$e[3,3] = 0.25$$

$$\text{root}[3,3] = 3$$

for $x=4$ to 4
 $t = e[4,3] + e[5,4] + w[4,4]$
 $t = 0.05 + 0.05 + 0.20 = 0.30$

$$0.30 < \infty$$

$$e[4,4] = 0.30$$

$$\text{root}[4,4] = 4$$

$i=1, j=1$
 $\delta=1$
 $1, 1$
 0.25
 5 to 5 i, j, δ
 $[5, 4] + e[6, 5] + w[5, 5]$
 $0.05 + 0.10 + 0.35 = 0.50$
 DATE:
 DAY:

$$e[5, 5] = 0.50$$

$$w[5, 5] = 5$$

Iteration 2

$$l=2$$

$$i=1 \text{ to } 4$$

$$e[1, 2] = \infty$$

$$w[1, 2] = w[1, 1] + P_2 + v_2$$

$$w[1, 2] = 0.30 + 0.10 + 0.05 = 0.45$$

for $\delta = 1$ to 2

$$t = e[1, 0] + e[2, 2] + w[1, 2]$$

$i=1$
 $j=2$
 $\delta=1$

$$t = 0.05 + 0.45 + 0.45 = 0.95$$

$$0.95 < 0.50 \text{ false}$$

$$e[1, 2] = 0.90 \quad \delta=2$$

$$t = e[1, 1] + e[3, 2] + w[1, 2]$$

$i=1$
 $j=1$

$$= 0.45 + 0.05 + 0.45$$

$$= 0.95 < 0.90 \text{ false}$$

$$e[1, 2] =$$

$\Rightarrow$ Now $i=2, j=3$

$$e[2, 3] = \infty$$

$$w[2, 3] = w[2, 2] + P_3 + v_3$$

$$w[2, 3] = 0.25 + 0.05 + 0.05 = 0.35$$

for $\delta = 2$ to 3

$i=2$
 $j=3$
 $\delta=2$

$$t = e[2, 1] + e[3, 3] + w[2, 3]$$

$$= 0.10 + 0.25 + 0.35 = 0.70$$

$$0.70 < \infty$$

$$e[2,3] = 0.70$$

$$\text{root}[2,3] = 2$$

DATE: $i=2$
 $j=3$
 $\delta=3$

$$t = e[2,2] + e[4,3] + w[2,3]$$

$$= 0.40 + 0.05 + 0.35$$

$$= 0.80$$

$$0.80 < 0.70 \text{ false}$$

$\Rightarrow$ Now $i=3, j=4$

$$e[3,4] = \infty$$

$$w[3,4] = w[3,3] + p_4 + v_4$$

$$w[3,4] = 0.15 + 0.10 + 0.05$$

$$= 0.30$$

~~$0.30 < 0.80$ false~~

~~$e[3,4] = 0.30$~~

for $\delta = 3$ to 4

$\delta=3$
 $i=3$
 $j=4$

$$t = e[3,2] + e[4,4] + w[3,4]$$

$$= 0.05 + 0.30 + 0.30$$

$$= 0.65$$

$$e[3,4] = 0.65$$

$$t = e[3,3] + e[5,4] + w[3,4]$$

$$= 0.25 + 0.05 + 0.30$$

$$= 0.60$$

$$0.60 < 0.65$$

$$e[3,4] = 0.60$$

root[3,4] DAY: 7

$$\Rightarrow \text{Now } i=4, j=5$$

$$e[4,5] = \infty$$

$$w[4,5] = w[4,4] + p_5$$

$$w[4,5] = 0.20 + 0.20 + 0.10$$

$$w[4,5] = 0.50$$

for $\delta = 4$ to 5

$\delta=4$
 $i=4$
 $j=5$

$$t = e[4,4] + e[5,5] + w[4,5]$$

$$= 0.30 + 0.50 + 0.50$$

$$t = 1.30$$

$$1.30 < \infty$$

$$e[4,5] = 1.30$$

$\delta=5$
 $i=4$
 $j=5$

$$t = e[4,4] + e[6,5] + w[4,5]$$

$$t = 0.30 + 0.10 + 0.50$$

$$= 0.90$$

$$e[4,5] = 0.90$$

$$0.90 < 1.30$$

$$e[4,5] = 0.90$$

$$\text{root}[4,5] = 5$$

tion 31 $X=3$

for $i=1$ to 3
 $e[1,3] = \infty$

DATE:

$$w[1,3] = w[1,2] + p_3 + cv_3$$

$$w[1,3] = 0.45 + 0.05 + 0.05$$

$$w[1,3] = 0.55$$

for $\delta=1$ to 3

$\delta=1$

$i=1$

$j=3$

$$t = e[1,0] + e[2,3] + w[1,3]$$

$$t = 0.05 + 0.70 + 0.55$$

$$= 0.60 + 0.70$$

$$t = 1.30$$

$$1.30 < \infty \text{ true}$$

$$e[1,3] = 1.30$$

$$\text{root}[1,3] = 1$$

$\delta=2$

$i=1$

$j=3$

$$t = e[1,1] + e[3,3] + w[1,3]$$

$$= 0.45 + 0.25 + 0.55$$

$$= 1.25$$

$$1.25 < 1.30 \text{ True}$$

$$e[1,3] = 1.25$$

$$\text{root}[1,3] = 2$$

$\delta=3$

$$t = e[1,2] + e[4,3] + w[1,3]$$

$$= 0.90 + 0.05 + 0.55$$

$$= 0.90 + 0.60 = 1.50$$

$$1.50 < 1.25 \text{ false}$$

$$\Rightarrow \text{Now, } i=2, j=4$$

$$e[2,4] = \infty$$

$$w[2,4] = w[2,3] + p_4 + cv_4$$

$$w[2,4] = 0.35 + 0.10 + 0.05$$

$$w[2,4] = 0.50$$

for $\delta=2$ to 4

$i=2$

$j=4$

$\delta=2$

$$t = e[2,1] + e[3,4] + w[2,4]$$

$$t = 0.10 + 0.60 + 0.50$$

$$= 1.20$$

$$1.20 < \infty \text{ True}$$

$$e[2,4] = 1.20$$

$$\text{root}[2,4] = 2$$

$i=2$

$j=4$

$$t = e[2,2] + e[4,4] + w[2,4]$$

$$t = 0.40 + 0.30 + 0.50$$

$$= 1.20$$

$$1.20 < 1.20 \text{ false}$$

$\delta=4$

$i=2$

$j=4$

$$t = e[2,3] + e[5,4] + w[2,4]$$

$$= 0.70 + 0.05 + 0.50$$

$$= 0.70 + 0.55$$

$$= 1.25$$

$$1.25 < 1.20 \text{ false}$$

now, $i=3, j=5$

$e[3,5] = \infty$

$w[3,5] = w[3,4] + p[3,5]$

$= 0.30 + 0.20 + 0.10$

$w[3,5] = 0.60$

for $x=3$ to 5

$t = e[3,2] + e[4,5] \quad x=3$

$+ w[3,5] \quad i=3$

$j=5$

$= 0.05 + 0.90 + 0.60$

$= 1.55$

$1.55 < \infty$ true

$e[3,5] = 1.55$

root[3,5] = 3

$x=4$

$i=3$

$j=5$

$t = e[3,3] + e[4,5] + w[3,5]$

$= 0.25 + 0.90 + 0.60$

$= 1.75$

$1.75 < 1.55$ false

$x=5$

$i=3$

$j=5$

$t = e[3,4] + e[5,5] + w[3,5]$

$= 0.60 + 0.10 + 0.60$

$= 1.30$

$1.30 < 1.55$ true

$e[3,5] = 1.30$

root[3,5] = 5

Iteration 4

$l=4$

for $i=2$ to 2

$j=4$

$e[1,4] = \infty$

$w[1,4] = w[1,3] + p[1,4]$

$= 0.55 + 0.10 + 0.05$

$w[1,4] = 0.70$

for $x=1$ to 4

$t = e[1,0] + e[2,4] \quad x=1$

$+ w[1,4] \quad i=1$

$j=4$

$= 0.05 + 1.20 + 0.70$

$= 0.75 + 1.20 = 1.95$

$1.95 < \infty$ true

$e[1,4] = 1.95$

root[1,4] = 1

$x=2$

$i=1$

$j=4$

$t = e[1,1] + e[3,4] + w[1,4]$

$= 0.45 + 0.60 + 0.70$

$= 1.30 + 0.45 = 1.75$

$1.75 < 1.95$ true

$e[1,4] = 1.75$

root[1,4] = 2

$$e(1,2) + e(4,4) = 4$$

$$w(1,4)$$

$$1.0 + 0.30 + 0.70$$

$$1.70$$

$$1.70 < 1.75 \text{ false}$$

$$i=4$$

$$j=1$$

$$j=4$$

$$e(1,3) + e(5,4) + w(1,4)$$

$$1.25 + 0.05 + 0.70$$

$$1.30 + 0.70 = 2.00$$

$$2.00 < 1.75 \text{ false}$$

$$\Rightarrow \text{Now } i=2, j=5$$

$$e(2,5) = \infty$$

$$w(2,5) = w(2,4) + P_5 + v_5$$

$$= 0.50 + 0.20 + 0.10$$

$$w(2,5) = 0.80$$

$$\text{for } i=2 \text{ to } 5$$

$$t = e(2,i) + e(3,5) + w(2,5)$$

$$i=2$$

$$j=5$$

$$= 0.10 + 1.20 + 0.80$$

$$= 2.10$$

$$2.10 < \infty \text{ True}$$

$$e(2,5) = 2.10$$

$$\text{root}(2,5) = 2$$

$$e(2,2) + e(4,5) + w(2,5)$$

$$= 0.40 + 0.70 + 0.80$$

$$1.20 + 0.70 = 1.90$$

$$1.90 < 2.10 \text{ false}$$

$$i=2$$

$$j=2$$

$$j=5$$

$$t = e(2,3) + e(5,5) + w(2,5)$$

$$t = 0.70 + 0.50 + 0.80$$

$$= 1.80 + 0.80$$

$$= 2.00$$

$$2.00 < 2.10$$

$$\text{True}$$

$$e(2,5) = 2.00$$

$$\text{root}(2,5) = 4$$

$$i=5$$

$$j=2$$

$$j=5$$

$$t = e(2,4) + e(5,5) + w(2,5)$$

$$= 1.20 + 0.70 + 0.80$$

$$= 2.70$$

$$2.70 < 2.00 \text{ false}$$

Iteration 5:

$$l=5$$

for $i=1$ to l

$$j = l+1$$
$$j=5$$

$$e[1,5] = \infty$$

$$w[1,5] = w[1,4] + p_5 + v_5$$
$$= 0.70 + 0.20 + 0.10$$

$$w[1,5] = 1.00$$

for $i=1$ to 5

$$x=1, i=1, j=5$$

$$t = e[1,0] + e[2,5] + w[1,5]$$
$$= 0.05 + 2.00 + 1.00$$
$$= 3.05$$

$$3.05 < \infty \text{ true}$$

$$e[1,5] = 3.05$$

$$\text{root}[1,5] = 1$$

$$x=2, i=1, j=5$$

$$t = e[1,1] + e[3,5] + w[1,5]$$
$$= 0.45 + 1.30 + 1.00$$
$$= 2.75$$

$$2.75 < 3.05 \text{ True}$$

$$e[1,5] = 2.75$$

$$\text{root}[1,5] = 2$$

$$x=3, i=1, j=5$$

$$t = e[1,2] + e[4,5] + w[1,5]$$
$$= 0.90 + 0.90 + 1.00$$
$$= 1.80 + 1.00$$
$$= 2.80$$

$$2.80 < 2.75 \text{ false}$$

$$t = e[1,3] + e[5,5] + w[1,5]$$

DAY:

$$= 1.25 + 0.50 + 1.00$$
$$= 1.75 + 1.00$$
$$= 2.75$$

$$2.75 < 2.75 \text{ false}$$

$$x=5$$

$$i=1$$

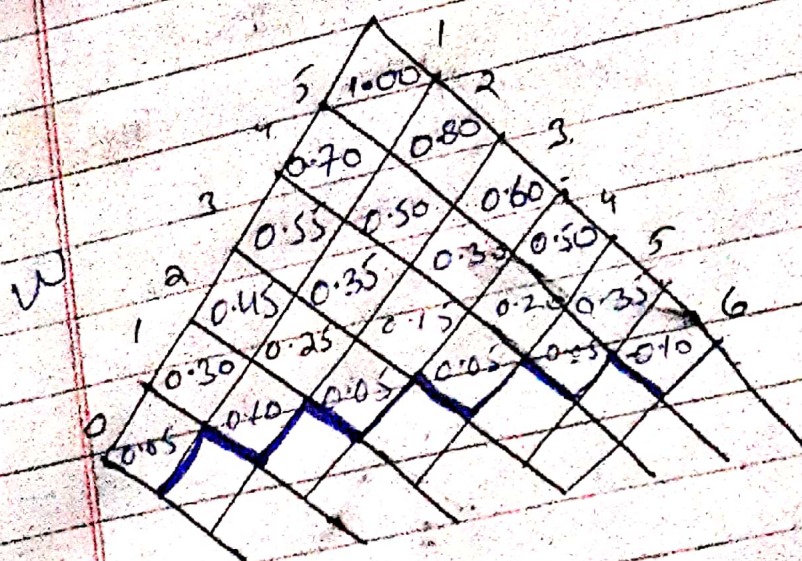
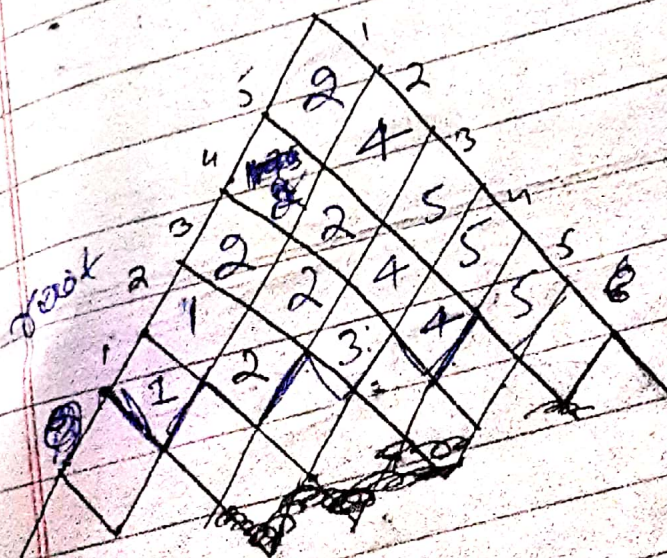
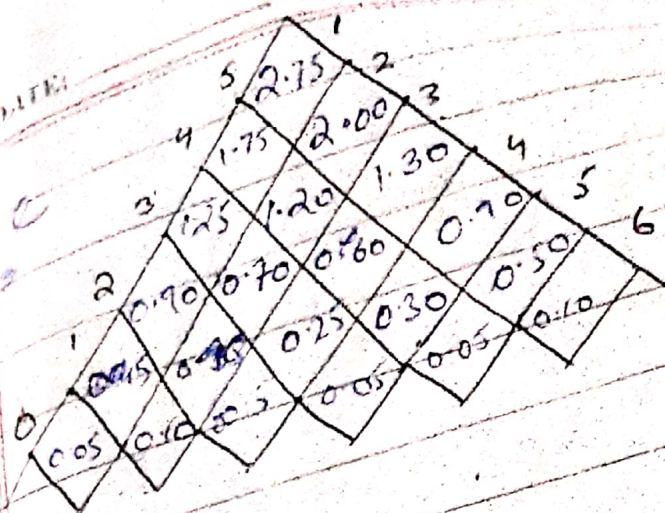
$$j=5$$

$$t = e[1,4] + e[5,5] + w[1,5]$$

$$= 1.75 + 0.10 + 1.00$$
$$= 1.85 + 1.00$$
$$= 2.85$$

$$2.85 < 2.75 \text{ false}$$

DATE



DATE: _____ DAY: _____

Constant - Optimal - BST (root)

$x = \text{root}[1, n]$

Point k_x is the root.

Constant - Optimal - subtree $(1, x-1, x,$
"left", root)

constant - optimal - subtree $(x+1, n, x,$
"right", root)

Constant - Optimal - subtree $(i, j, x,$
dis, root)

if $i \leq j$
 $t = \text{root}[i, j]$

point k_t "is" the dis "child" of

constant - optimal subtree $(1, t-1, t,$
"left", root)

Constant - optimal - subtree $(t+1, j, t,$
"right", root)